DATA 260 - Homework 8

Name - Ayushi Shringi
GitHub repository - https://github.com/ayushishringi/data260-0036
SID4 - 0036
PORT_BASE - 8036 (8000 + 36 mod 900)
PREFIX - s0036
SEED - 36
VERIFY_SEED - 260036
DOMAIN_ID - 4 (36 mod 8): open-source package vulnerability tracking
Hardware - MacBook Pro, Apple M5 Pro, 24 GB
Local model - Ollama qwen3:4b (num_ctx 16384), used by the Planner and Writer
Kafka - apache/kafka:3.9.0, KRaft single broker (Docker)
Tag - hw8
Commit - 6a17a98

The GitHub link was verified, and both collaborators (Sbnikitha, supriyaselvanganesan) have access.

Part 1 - Backend Service Request/Response over Kafka

The primary entity is VulnerabilityReport, so CREATE_ENTITY CREATE_REPORT, GET_ENTITY GET_REPORT, entity_id report_id. The two topics are s0036_report_requests (client/API worker) and s0036_report_responses (worker client/API). The worker keeps reports in a plain Python dict and generates report_id with uuid4.

Message schemas

Request: {"correlation_id": "<uuid>", "reply_to": "s0036_report_responses",
"operation": "CREATE_REPORT", "data": {"package_name", "affected_version",
"severity", "description", "submitter_email"}}
{"correlation_id": "<uuid>", "reply_to": "s0036_report_responses",
"operation": "GET_REPORT", "data": {"report_id": "<uuid>"}}
Success: {"correlation_id", "success": true, "report_id", "message": "Report created"}
{"correlation_id", "success": true, "report": {"report_id", fields}}
Error: {"correlation_id", "success": false,
"error": "Report not found" | "Unknown operation" | <validation message>}

```
    REQUEST_TOPIC = f"{PREFIX}_report_requests"
    RESPONSE_TOPIC = f"{PREFIX}_report_responses"
    # Part 2/3: three-agent pipeline.
    INBOX_TOPIC = f"{PREFIX}_inbox"
    TASKS_TOPIC = f"{PREFIX}_tasks"
    DRAFTS_TOPIC = f"{PREFIX}_drafts"
    FINAL_TOPIC = f"{PREFIX}_final"
    DEAD_LETTER_TOPIC = f"{PREFIX}_dead_letter"
```

```
def validate_report(data: Any) -> tuple[dict[str, str] | None, str | None]:
    """Return (clean_fields, None) or (None, error) for CREATE_REPORT data."""

    if not isinstance(data, dict):
        return None, "data must be a JSON object"
    missing = [f for f in REQUIRED_FIELDS if f not in data]
    if missing:
        return None, f"Missing required field(s): {', '.join(missing)}"
    unknown = sorted(set(data) - set(REQUIRED_FIELDS))
    if unknown:
        return None, f"Unknown field(s): {', '.join(unknown)}"
    clean: dict[str, str] = {}
    for field in REQUIRED_FIELDS:
        value = data[field]
        if not isinstance(value, str) or not value.strip():
            return None, f"Field '{field}' must be a non-empty string"
        clean[field] = value.strip()
    if len(clean["package_name"]) > 100:
        return None, "Field 'package_name' must be at most 100 characters"
    if clean["severity"].lower() not in SEVERITIES:
        return None, f"Field 'severity' must be one of: {', '.join(SEVERITIES)}"
    clean["severity"] = clean["severity"].lower()
    if len(clean["description"]) < 20:
        return None, "Field 'description' must be at least 20 characters"
    if not EMAIL_RE.match(clean["submitter_email"]):
        return None, "Field 'submitter_email' must be a valid email address"
    return clean, None


def handle_request(message: Any, store: dict[str, dict[str, Any]]) -> dict[str, Any]:
    """Process one request message and return the response (always echoing correlation_id)."""

    correlation_id = message.get("correlation_id") if isinstance(message, dict) else None
    base = {"correlation_id": correlation_id}
    if not isinstance(message, dict):
        return {**base, "success": False, "error": "Request must be a JSON object"}
    missing = [k for k in REQUEST_KEYS if k not in message]
    if missing:
        return {**base, "success": False, "error": f"Malformed request: missing {', '.join(missing)}"}
    operation, data = message["operation"], message["data"]
    base["operation"] = operation
```

```
    base["operation"] = operation

    if operation == CREATE_OPERATION:
        clean, error = validate_report(data)
        if error:
            return {**base, "success": False, "error": error}
        report_id = str(uuid.uuid4())
        store[report_id] = {"report_id": report_id, **clean}
        return {**base, "success": True, "report_id": report_id, "message": "Report created"}

    if operation == GET_OPERATION:
        report_id = data.get("report_id") if isinstance(data, dict) else None
        if not isinstance(report_id, str) or not report_id.strip():
            return {**base, "success": False, "error": "Field 'report_id' is required"}
        report = store.get(report_id.strip())
        if report is None:
            return {**base, "success": False, "error": "Report not found"}
        return {**base, "success": True, "report": dict(report)}

    return {**base, "success": False, "error": "Unknown operation"}
```

```python
def main() -> None:
    wait_for_kafka()
    store: dict[str, dict] = {}  # plain in-memory dictionary, as required for this exercise
    consumer = make_consumer("s0036_report_workers")
    producer = make_producer()
    announced = False

    def on_assign(_consumer: object, partitions: list) -> None:
        nonlocal announced
        if not announced:  # logged from the rebalance callback, so no message is polled and dropped first
            log(COMPONENT, "Consumer is ready and listening...", topic=REQUEST_TOPIC,
                partitions=[p.partition for p in partitions])
            announced = True

    consumer.subscribe([REQUEST_TOPIC], on_assign=on_assign)

    running = True

    def stop(*_args: object) -> None:
        nonlocal running
        running = False

    signal.signal(signal.SIGTERM, stop)
    signal.signal(signal.SIGINT, stop)
    while running:
        msg = consumer.poll(1.0)
        if msg is None or msg.error():
            continue
        try:
            request = decode(msg)
        except ValueError as exc:
            log(COMPONENT, "DROPPED unparseable request", offset=msg.offset(), error=str(exc))
            consumer.commit(message=msg, asynchronous=False)
            continue
        cid = request.get("correlation_id")
        log(COMPONENT, "RECEIVED request", topic=msg.topic(), partition=msg.partition(), offset=msg.offset(),
            correlation_id=cid, operation=request.get("operation"), data=request.get("data"))
        response = handle_request(request, store)
        log(COMPONENT, "PROCESSED request", correlation_id=cid, success=response["success"],
            result=response.get("report_id") or response.get("error") or "report returned")
        reply_to = request.get("reply_to") if isinstance(request.get("reply_to"), str) else None
        if reply_to:
            log(COMPONENT, "SENT response", topic=reply_to, correlation_id=cid, response=response)
```

```
            request = decode(msg)
        except ValueError as exc:
            log(COMPONENT, "DROPPED unparseable request", offset=msg.offset(), error=str(exc))
            consumer.commit(message=msg, asynchronous=False)
            continue
        cid = request.get("correlation_id")
        log(COMPONENT, "RECEIVED request", topic=msg.topic(), partition=msg.partition(), offset=msg.offset(),
            correlation_id=cid, operation=request.get("operation"), data=request.get("data"))
        response = handle_request(request, store)
        log(COMPONENT, "PROCESSED request", correlation_id=cid, success=response["success"],
            result=response.get("report_id") or response.get("error") or "report returned")
        reply_to = request.get("reply_to") if isinstance(request.get("reply_to"), str) else None
        if reply_to:
            log(COMPONENT, "SENT response", topic=reply_to, correlation_id=cid, response=response)
            sent = produce_json(producer, reply_to, response, key=cid)
            log(COMPONENT, "response acknowledged by broker", topic=reply_to, correlation_id=cid, **sent)
        else:
            log(COMPONENT, "NO reply_to; response not sent", correlation_id=cid)
        consumer.commit(message=msg, asynchronous=False)  # commit only after the reply is acknowledged
    consumer.close()


if __name__ == "__main__":
    main()

```

```
class ReportRpcClient:
    """Thread-safe request/response client; one instance can serve many concurrent callers."""

    def __init__(self, component: str = "client") -> None:
        self.component = component
        self.producer = make_producer()
        self.consumer = make_consumer(f"s0036_reply_reader_{uuid.uuid4().hex[:8]}", offset_reset="latest")
        assign_at_end(self.consumer, RESPONSE_TOPIC)
        self._pending: dict[str, dict[str, Any]] = {}
        self._lock = threading.Lock()
        self._running = True
        self._thread = threading.Thread(target=self._read_replies, daemon=True)
        self._thread.start()

    def _read_replies(self) -> None:
        while self._running:
            msg = self.consumer.poll(0.5)
            if msg is None or msg.error():
                continue
            try:
                response = decode(msg)
            except ValueError:
                continue
            cid = response.get("correlation_id")
            with self._lock:
                slot = self._pending.get(cid)
            if slot is not None:  # responses for other callers are ignored
                slot["response"] = response
                slot["meta"] = {"partition": msg.partition(), "offset": msg.offset()}
                slot["event"].set()

    @staticmethod
    def new_correlation_id() -> str:
        return str(uuid.uuid4())
```

```python

    def request(self, operation: str, data: Any, correlation_id: str | None = None,
                timeout: float = 15.0) -> dict[str, Any]:
        """Send one request and wait for the response with the same correlation_id."""

        cid = correlation_id or self.new_correlation_id()
        message = {"correlation_id": cid, "reply_to": RESPONSE_TOPIC, "operation": operation,
        "data": data}
        slot: dict[str, Any] = {"event": threading.Event()}
        with self._lock:
            self._pending[cid] = slot
        try:
            log(self.component, "SENT request", topic=REQUEST_TOPIC, correlation_id=cid,
            message=message)
            sent = produce_json(self.producer, REQUEST_TOPIC, message, key=cid)
            log(self.component, "request acknowledged by broker", topic=REQUEST_TOPIC,
            correlation_id=cid, **sent)
            if not slot["event"].wait(timeout):
                log(self.component, "TIMEOUT waiting for response", correlation_id=cid,
                timeout_s=timeout)
                return {"correlation_id": cid, "success": False, "error": "Timed out waiting for
                worker"}
            response = slot["response"]
            log(self.component, "RECEIVED response", topic=RESPONSE_TOPIC, correlation_id=cid,
                correlation_id_preserved=response.get("correlation_id") == cid, response=response,
                **slot["meta"])
            return response
        finally:
            with self._lock:
                self._pending.pop(cid, None)

    def close(self) -> None:
        self._running = False
        self._thread.join(timeout=2)
        self.consumer.close()
```

```python
async def call(operation: str, data: Any, correlation_id: str | None) -> dict[str, Any]:
    client = state["client"]
    cid = correlation_id or client.new_correlation_id()
    log(COMPONENT, "HTTP request received", correlation_id=cid, operation=operation)
    response = await run_in_threadpool(client.request, operation, data, cid)
    log(COMPONENT, "HTTP response returned to caller", correlation_id=cid, success=response.get
    ("success"))
    return response


@app.get("/api/health")
def health() -> dict:
    return {"status": "ok", "port": PORT_BASE, "service": "s0036 report service over Kafka"}


@app.post("/api/kafka/reports")
async def create_report(data: dict[str, Any], x_correlation_id: str | None = Header(default=None))
-> dict:
    return await call(CREATE_OPERATION, data, x_correlation_id)


@app.get("/api/kafka/reports/{report_id}")
async def get_report(report_id: str, x_correlation_id: str | None = Header(default=None)) -> dict:
    return await call(GET_OPERATION, {"report_id": report_id}, x_correlation_id)


@app.post("/api/kafka/requests")
async def raw_request(body: RawRequest, x_correlation_id: str | None = Header(default=None)) -> dict:
    return await call(body.operation, body.data, x_correlation_id)
```

```python
def worker_lines(correlation_id: str, since: str) -> list[tuple[datetime, str]]:
    """The worker's log lines for one correlation_id, read from its container."""

    out = subprocess.run(["docker", "logs", "--since", since, WORKER_CONTAINER],
    capture_output=True, text=True)
    lines = []
    for line in (out.stdout + out.stderr).splitlines():
        if correlation_id in line and "[worker]" in line:
            lines.append((datetime.fromisoformat(line.split(" ", 1)[0]), line))
    return lines


def run_step(client: ReportRpcClient, title: str, operation: str, data: dict) -> dict:
    print(f"\n{'=' * 100}\n{title}\n{'=' * 100}")
    correlation_id = client.new_correlation_id()
    print(f"[step] created correlation_id = {correlation_id}")
    since = datetime.now().astimezone().isoformat()
    client_records: list[tuple[datetime, str]] = []
    original_log = log

    def capture(component: str, event: str, **fields: object) -> dict:
        record = original_log(component, event, **fields)
        detail = " ".join(f"{k}={json.dumps(v, default=str)}" for k, v in fields.items())
        client_records.append((datetime.fromisoformat(record["ts"]), f"{record['ts']} [{component}]
        {event} {detail}"))
        return record

    import src.kafka_rpc as rpc_module
    rpc_module.log = capture
    try:
        response = client.request(operation, data, correlation_id)
    finally:
        rpc_module.log = original_log
    time.sleep(0.5)  # let the worker's log reach `docker logs`
    labels = {"SENT request": "1 SEND    ", "RECEIVED request": "2 RECEIVE ", "PROCESSED request":
    "3 PROCESS ",
              "SENT response": "4 REPLY   ", "RECEIVED response": "5 CLIENT  "}

    def stage(line: str) -> int:
        return next((i for i, k in enumerate(labels) if f"] {k} " in line), 9)
```

```
    # time order; lines logged in the same millisecond keep pipeline-stage order
    sequence = sorted(client_records + worker_lines(correlation_id, since),
                      key=lambda item: (item[0], stage(item[1])))
    print(f"\n--- full sequence for correlation_id {correlation_id} (client + worker, time-ordered)
    ---")
    for _, line in sequence:
        label = next((v for k, v in labels.items() if f"] {k} " in line), "          ")
        print(f"{label}| {line[:400]}")
    preserved = response.get("correlation_id") == correlation_id
    print(f"\nresponse: {json.dumps(response, indent=2)}")
    print(f"correlation_id preserved end to end: {preserved}")
    return response


def main() -> None:
    print(f"HW8 demo - request topic {REQUEST_TOPIC}, response topic {RESPONSE_TOPIC}")
    client = ReportRpcClient(component="client")
    try:
        created = run_step(client, f"STEP A: {CREATE_OPERATION}", CREATE_OPERATION, REPORT)
        if not created.get("success"):
            raise SystemExit(f"CREATE failed: {created}")
        report_id = created["report_id"]
        print(f"\n[step] extracted report_id from the {CREATE_OPERATION} response: {report_id}")
        fetched = run_step(client, f"STEP B: {GET_OPERATION} report_id={report_id}", GET_OPERATION,
                           {"report_id": report_id})
        print(f"\n{'=' * 100}\nRETRIEVED REPORT\n{'=' * 100}")
        print(json.dumps(fetched.get("report"), indent=2))
        print(f"report_id matches the created one: {fetched.get('report', {}).get('report_id') ==
        report_id}")
    finally:
        client.close()


if __name__ == "__main__":
    main()
```

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % clear
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
make kafka-topics
NAMES                IMAGE                STATUS                  PORTS
s0036-api            s0036-hw8:latest     Up 48 minutes           127.0.0.1:8036->8036/tcp
s0036-report-worker  s0036-hw8:latest     Up 48 minutes
s0036-kafka          apache/kafka:3.9.0   Up 11 hours (healthy)   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
docker exec s0036-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe | grep -E '^Topic: s0036'
Topic: s0036_final      TopicId: hspXL4MtRAqKP55wiV2eiQ PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s0036_report_responses   TopicId: 3FGwD5s2SAmHT85_ypiyCw PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s0036_drafts     TopicId: t-RZKRc4TrKoMwyxm64Wyw PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s0036_dead_letter        TopicId: XWu6IeeRRWm1LDsE2q5Fag PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s0036_inbox      TopicId: 9tZ_M_HIQ6G5fRsNrsE9Mw PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s0036_tasks      TopicId: WhT1iS53S_644EL4FDMHJA PartitionCount: 2       ReplicationFactor: 1    Configs:
Topic: s0036_report_requests    TopicId: NtoMCtY-R-6bBJDI_XbpXg PartitionCount: 1       ReplicationFactor: 1    Configs:
```

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % clear
docker logs s0036-report-worker 2>&1 | grep "ready and listening"
docker logs s0036-api 2>&1 | grep -E "started at|Uvicorn running"
2026-10-30T08:07:22.198-07:00 [worker] Consumer is ready and listening... topic="s0036_report_requests" partitions=[0]
2026-10-30T08:07:22.448-07:00 [api] FastAPI service started at http://127.0.0.1:8036
INFO:     Uvicorn running on http://0.0.0.0:8036 (Press CTRL+C to quit)
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

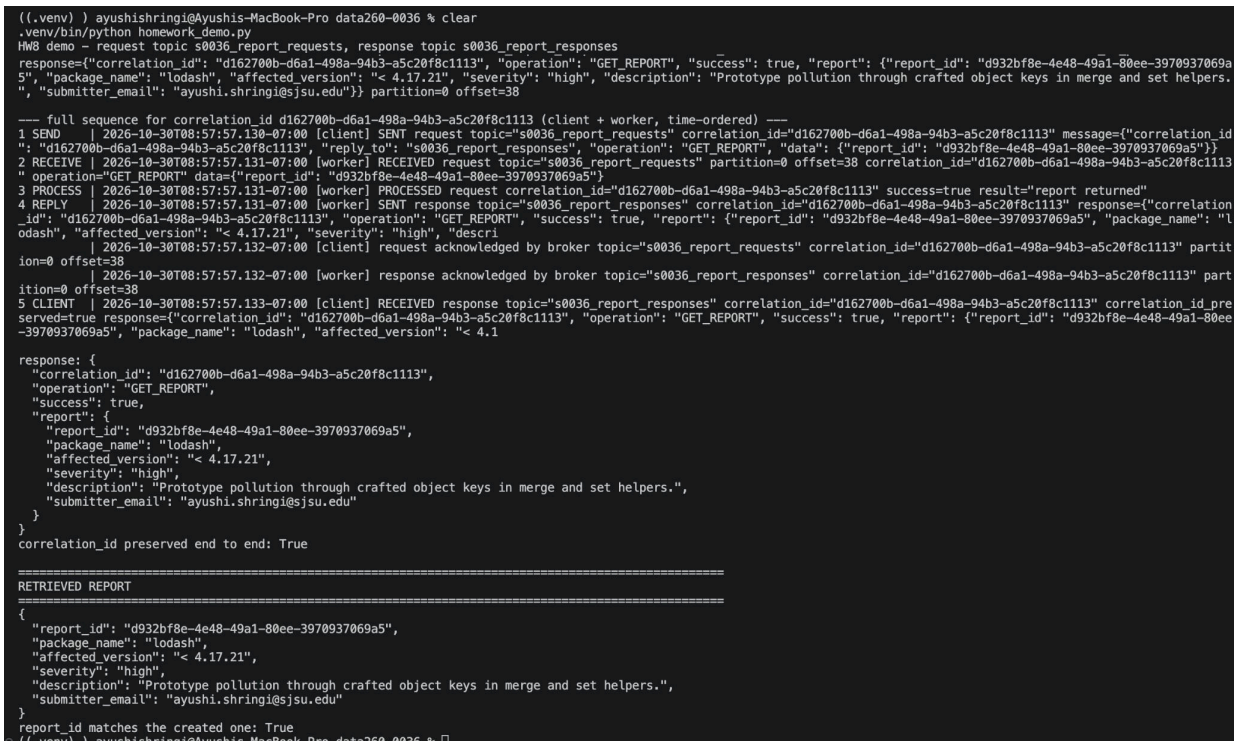
```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % clear
.venv/bin/python homework_demo.py
HW8 demo - request topic s0036_report_requests, response topic s0036_report_responses
================================================================================
STEP A: CREATE_REPORT
================================================================================
[step] created correlation_id = f657e979-6ee9-429c-8715-694185b77ff9
2026-10-30T08:57:55.588-07:00 [client] SENT request topic="s0036_report_requests" correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" message={"correlation_id": "f657e979
-6ee9-429c-8715-694185b77ff9", "reply_to": "s0036_report_responses", "operation": "CREATE_REPORT", "data": {"package_name": "lodash", "affected_version": "< 4.17.21", "seve
rity": "high", "description": "Prototype pollution through crafted object keys in merge and set helpers.", "submitter_email": "ayushi.shringi@sjsu.edu"}}
2026-10-30T08:57:56.591-07:00 [client] request acknowledged by broker topic="s0036_report_requests" correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" partition=0 offset
=37
2026-10-30T08:57:56.599-07:00 [client] RECEIVED response topic="s0036_report_responses" correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" correlation_id_preserved=true
response={"correlation_id": "f657e979-6ee9-429c-8715-694185b77ff9", "operation": "CREATE_REPORT", "success": true, "report_id": "d932bf8e-4e48-49a1-80ee-3970937069a5", "mes
sage": "Report created"} partition=0 offset=37

--- full sequence for correlation_id f657e979-6ee9-429c-8715-694185b77ff9 (client + worker, time-ordered) ---
1 SEND    | 2026-10-30T08:57:55.588-07:00 [client] SENT request topic="s0036_report_requests" correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" message={"correlation_id
": "f657e979-6ee9-429c-8715-694185b77ff9", "reply_to": "s0036_report_responses", "operation": "CREATE_REPORT", "data": {"package_name": "lodash", "affected_version": "< 4.1
7.21", "severity": "high", "description": "Prototype pollution throu
2 RECEIVE | 2026-10-30T08:57:56.591-07:00 [worker] RECEIVED request topic="s0036_report_requests" partition=0 offset=37 correlation_id="f657e979-6ee9-429c-8715-694185b77ff9
" operation="CREATE_REPORT" data={"package_name": "lodash", "affected_version": "< 4.17.21", "severity": "high", "description": "Prototype pollution through crafted object
keys in merge and set helpers.", "submitter_email": "ayushi.shringi@
          | 2026-10-30T08:57:56.591-07:00 [client] request acknowledged by broker topic="s0036_report_requests" correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" partit
ion=0 offset=37
3 PROCESS | 2026-10-30T08:57:56.595-07:00 [worker] PROCESSED request correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" success=true result="d932bf8e-4e48-49a1-80ee-3970
937069a5"
4 REPLY   | 2026-10-30T08:57:56.595-07:00 [worker] SENT response topic="s0036_report_responses" correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" response={"correlation
_id": "f657e979-6ee9-429c-8715-694185b77ff9", "operation": "CREATE_REPORT", "success": true, "report_id": "d932bf8e-4e48-49a1-80ee-3970937069a5", "message": "Report created
"}
          | 2026-10-30T08:57:56.597-07:00 [worker] response acknowledged by broker topic="s0036_report_responses" correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" part
ition=0 offset=37
5 CLIENT  | 2026-10-30T08:57:56.599-07:00 [client] RECEIVED response topic="s0036_report_responses" correlation_id="f657e979-6ee9-429c-8715-694185b77ff9" correlation_id_pre
served=true response={"correlation_id": "f657e979-6ee9-429c-8715-694185b77ff9", "operation": "CREATE_REPORT", "success": true, "report_id": "d932bf8e-4e48-49a1-80ee-3970937
069a5", "message": "Report created"} partition=0 offset=37

response: {
  "correlation_id": "f657e979-6ee9-429c-8715-694185b77ff9",
  "operation": "CREATE_REPORT",
  "success": true,
  "report_id": "d932bf8e-4e48-49a1-80ee-3970937069a5",
  "message": "Report created"
}
correlation_id preserved end to end: True

[step] extracted report_id from the CREATE_REPORT response: d932bf8e-4e48-49a1-80ee-3970937069a5

================================================================================
STEP B: GET_REPORT report_id=d932bf8e-4e48-49a1-80ee-3970937069a5
================================================================================
[step] created correlation_id = d162700b-d6a1-498a-94b3-a5c20f8c1113
2026-10-30T08:57:57.130-07:00 [client] SENT request topic="s0036_report_requests" correlation_id="d162700b-d6a1-498a-94b3-a5c20f8c1113" message={"correlation_id": "d162700b
```





The client creates one correlation_id per request. It travels unchanged in the request, the worker logs, the response and the client log, and the demo prints *correlation_id preserved end to end: True* for both CREATE_REPORT and GET_REPORT. The worker commits each request offset only after its reply is acknowledged by Kafka.

Part 2 - Three-Agent Kafka Pipeline

Topics: s0036_inbox Planner s0036_tasks (2 partitions) Writer s0036_drafts Reviewer s0036_final. Each agent is a separate process that only knows its input and output topic.

```
198
199
200   def planner_handle(message: dict[str, Any]) -> dict[str, Any]:
201       question = message.get("question")
202       if not isinstance(question, str) or not question.strip():
203           raise PoisonMessage("question must be a non-empty string")
204       if use_llm():
205           text = ask_model("You plan answers about open-source package vulnerabilities. Reply with
          exactly 3 short "
206                            "numbered steps a writer should follow. No other text.", question)
207           plan = [re.sub(r"^\s*\d+[.)]\s*", "", line).strip() for line in text.splitlines() if line.
          strip()][:3]
208       else:
209           plan = [f"Define the key terms in: {question}", "Explain the main risk with one concrete
          example",
210                   "Give two practical mitigation steps"]
211       return {"correlation_id": message["correlation_id"], "kind": "task", "question": question,
          "plan": plan,
212               "planned_at": now_iso()}
213
214
```

```
def writer_handle(message: dict[str, Any]) -> dict[str, Any] | None:
    if message.get("kind") == "ordering_probe":
        # Part 3 ordering test: simulated work whose duration is fixed per message (seeded).
        seed = int(hashlib.sha256(message["probe_id"].encode()).hexdigest(), 16) % 10_000
        time.sleep(random.Random(seed).uniform(0.05, 0.6))
        log(f"writer#{env('HW8_INSTANCE', 'writer')}", "PROCESSED ordering probe",
            probe_id=message["probe_id"], entity=message["entity"], seq=message["seq"],
            sent_index=message["index"])
        return None
    plan, question = message.get("plan"), message.get("question")
    if not isinstance(plan, list) or not plan or not all(isinstance(step, str) for step in plan):
        raise PoisonMessage("plan must be a non-empty list of strings")
    if not isinstance(question, str):
        raise PoisonMessage("question must be a string")
    if use_llm():
        draft = ask_model("You write concise, accurate answers (120-180 words) about open-source
        package "
                          "vulnerabilities. Follow the plan.",
                          f"Question: {question}\nPlan:\n" + "\n".join(f"- {s}" for s in plan))
    else:
        draft = f"Answer to '{question}': " + " ".join(f"({i}) {step}." for i, step in enumerate
        (plan, 1))
    return {"correlation_id": message["correlation_id"], "kind": "draft", "question": question,
    "plan": plan,
            "draft": draft, "written_at": now_iso()}
```

```python
def reviewer_handle(message: dict[str, Any]) -> dict[str, Any]:
    draft, question = message.get("draft"), message.get("question")
    if not isinstance(draft, str) or not isinstance(question, str):
        raise PoisonMessage("draft and question must be strings")
    words = len(draft.split())
    keywords = {w for w in re.findall(r"[a-z]{5,}", question.lower())}
    covered = sum(1 for w in keywords if w in draft.lower())
    checks = {"non_empty": words > 0, "length_ok": 15 <= words <= 400,
              "on_topic": covered >= max(1, len(keywords) // 3)}
    status = "approved" if all(checks.values()) else "needs_revision"
    return {"correlation_id": message["correlation_id"], "status": status, "answer": draft,
    "question": question,
            "review": {"checks": checks, "words": words}, "reviewed_at": now_iso()}


AGENTS = {
    "planner": (INBOX_TOPIC, TASKS_TOPIC, planner_handle),
    "writer": (TASKS_TOPIC, DRAFTS_TOPIC, writer_handle),
    "reviewer": (DRAFTS_TOPIC, FINAL_TOPIC, reviewer_handle),
}
```

```python
def run_agent(name: str, in_topic: str, out_topic: str | None,
              handle: Callable[[dict[str, Any]], dict[str, Any] | None]) -> None:
    """Consume in_topic, call handle(message), produce its result to out_topic, then commit."""

    wait_for_kafka()
    instance = env("HW8_INSTANCE", name)
    group = f"{PREFIX}_{name}s{env('HW8_GROUP_SUFFIX', '')}"
    idempotent = env("HW8_IDEMPOTENT", "1") == "1"
    commit_mode = env("HW8_COMMIT", "after")
    retry_mode = env("HW8_RETRY", "bounded")
    max_retries = int(env("HW8_MAX_RETRIES", "3"))
    slow_seconds = float(env("HW8_SLOW_SECONDS", "0"))
    component = f"{name}#{instance}" if instance != name else name
    store = ProcessedStore(group) if idempotent else None
    consumer = make_consumer(group)
    producer = make_producer()

    def on_assign(c: Any, partitions: list[TopicPartition]) -> None:
        # A brand-new group starts at the current end of the topic (so old experiments are not
        # replayed); a group that has committed offsets resumes exactly where it committed.
        committed = c.committed(partitions, timeout=10)
        start = []
        for tp in committed:
            if tp.offset < 0:
                _low, high = c.get_watermark_offsets(TopicPartition(tp.topic, tp.partition),
                timeout=10)
                start.append(TopicPartition(tp.topic, tp.partition, high))
            else:
                start.append(tp)
        c.assign(start)
        c.commit(offsets=start, asynchronous=False)  # record the starting point for a new group
        log(component, "Consumer is ready and listening...", topic=in_topic, group=group,
            partitions={tp.partition: tp.offset for tp in start}, idempotent=idempotent,
            commit=commit_mode,
            retry=retry_mode, max_retries=max_retries)

    consumer.subscribe([in_topic], on_assign=on_assign)
    running = True
```

```
    def stop(*_args: object) -> None:
        nonlocal running
        running = False

    signal.signal(signal.SIGTERM, stop)
    signal.signal(signal.SIGINT, stop)

    while running:
        msg = consumer.poll(1.0)
        if msg is None or msg.error():
            continue
        where = {"partition": msg.partition(), "offset": msg.offset()}
        if commit_mode == "before":  # BEFORE FIX: offset committed as soon as the message is read
            consumer.commit(message=msg, asynchronous=False)
            log(component, "COMMITTED offset on receipt (before processing)", **where)

        attempt = 0
        while True:
            attempt += 1
            try:
                message = decode(msg)
            except (ValueError, UnicodeDecodeError) as exc:
                message, error = None, PoisonMessage(f"not valid JSON: {exc}")
            else:
                error = None
            cid = message.get("correlation_id") if isinstance(message, dict) else None
            try:
                if error:
                    raise error
                if attempt == 1:
                    log(component, "RECEIVED", topic=in_topic, correlation_id=cid, **where,
                        kind=message.get("kind", "question"))
                if store is not None and cid and store.seen(cid):
                    log(component, "DUPLICATE skipped (already processed)", correlation_id=cid,
                    **where)
                    output = None
                    break
                if slow_seconds:
                    log(component, "STARTED long processing", correlation_id=cid,
                    seconds=slow_seconds, **where)
                    time.sleep(slow_seconds)
                output = handle(message)
                if output is not None and out_topic:
```

```
                if slow_seconds:
                    log(component, "STARTED long processing", correlation_id=cid,
                    seconds=slow_seconds, **where)
                    time.sleep(slow_seconds)
                output = handle(message)
                if output is not None and out_topic:
                    log(component, "SENT", topic=out_topic, correlation_id=cid,
                        **{k: v for k, v in output.items() if k in ("plan", "draft", "status",
                        "answer")})
                    produce_json(producer, out_topic, output, key=output.get("correlation_id"))
                if store is not None and cid:
                    store.mark(cid)
                break
            except Exception as exc:  # poison or transient failure
                log(component, "PROCESSING FAILED", correlation_id=cid, attempt=attempt, error=str
                (exc), **where)
                if retry_mode == "unbounded":  # BEFORE FIX: retry forever, blocking the partition
                    time.sleep(1.0)
                    continue
                if attempt < max_retries:
                    time.sleep(0.5 * attempt)
                    continue
                dead = {"correlation_id": cid, "failed_agent": name, "source_topic": in_topic,
                **where,
                        "attempts": attempt, "error": str(exc), "ts": now_iso(),
                        "original": message if message is not None else msg.value().decode("utf-8",
                        "replace")}
                produce_json(producer, DEAD_LETTER_TOPIC, dead, key=cid)
                log(component, "MOVED to dead-letter topic", topic=DEAD_LETTER_TOPIC,
                correlation_id=cid,
                    attempts=attempt, **where)
                break
        if commit_mode == "after":  # AFTER FIX: commit only once the output is safely in Kafka
            consumer.commit(message=msg, asynchronous=False)
            log(component, "COMMITTED offset after processing", correlation_id=cid, **where)
    consumer.close()
```

```python
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("question", nargs="?", default=DEFAULT_QUESTION)
    parser.add_argument("--replay", action="store_true", help="resend the last message unchanged")
    parser.add_argument("--timeout", type=float, default=300)
    parser.add_argument("--no-wait", action="store_true")
    args = parser.parse_args()

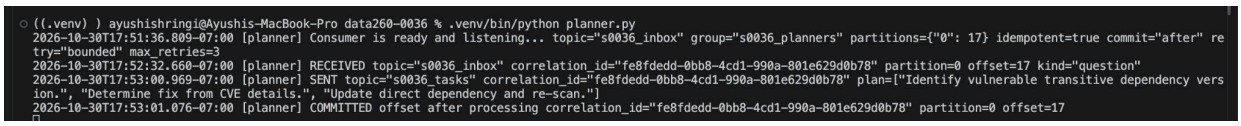
    if args.replay:
        message = json.loads(LAST.read_text())
        log("sender", "REPLAYING previous message with the same correlation_id",
            correlation_id=message["correlation_id"])
    else:
        message = {"correlation_id": str(uuid.uuid4()), "kind": "question", "question": args.question,
                   "asked_at": now_iso()}
        STATE_DIR.mkdir(parents=True, exist_ok=True)
        LAST.write_text(json.dumps(message))

    reader = make_consumer(f"s0036_final_reader_{uuid.uuid4().hex[:8]}", offset_reset="latest")
    assign_at_end(reader, FINAL_TOPIC)  # read everything produced to s0036_final from now on
    log("sender", "SENT question", topic=INBOX_TOPIC, correlation_id=message["correlation_id"],
        question=message["question"])
    produce_json(make_producer(), INBOX_TOPIC, message, key=message["correlation_id"])
    if args.no_wait:
        return
    deadline = time.time() + args.timeout
    while time.time() < deadline:
        msg = reader.poll(1.0)
        if msg is None or msg.error():
            continue
        final = decode(msg)
        if final.get("correlation_id") == message["correlation_id"]:
            log("sender", "RECEIVED final", topic=FINAL_TOPIC, correlation_id=final["correlation_id"],
                partition=msg.partition(), offset=msg.offset(), status=final.get("status"))
            print(json.dumps(final, indent=2))
            return
    log("sender", "NO final message before timeout (expected after a replay with idempotency on)",
        correlation_id=message["correlation_id"], timeout_s=args.timeout)

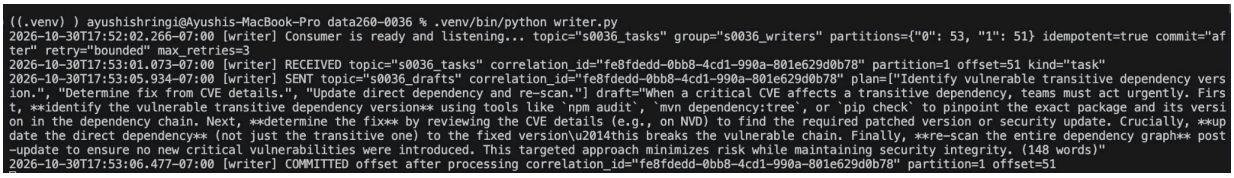

if __name__ == "__main__":
    main()
```

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % clear
docker logs s0036-kafka-init 2>&1 | grep -E "Created topic|already exists"
docker exec s0036-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic s0036_tasks
Created topic s0036_report_requests.
Created topic s0036_report_responses.
Created topic s0036_inbox.
Created topic s0036_tasks.
Created topic s0036_drafts.
Created topic s0036_final.
Created topic s0036_dead_letter.
Topic: s0036_tasks      TopicId: WhT1iS53S_644EL4FDMHJA PartitionCount: 2       ReplicationFactor: 1    Configs:
        Topic: s0036_tasks      Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: s0036_tasks      Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

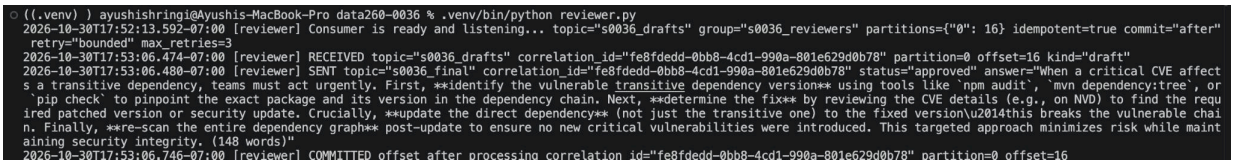
Topics created by kafka-init; s0036_tasks has PartitionCount 2.



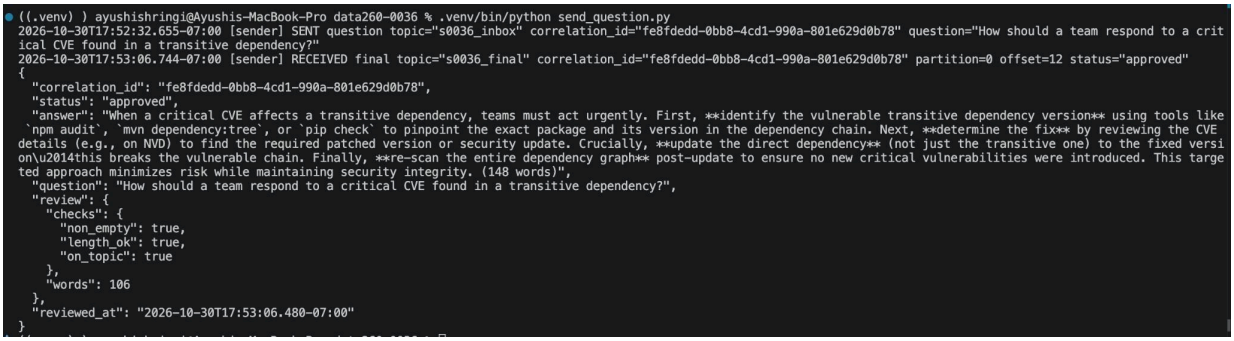
Terminal 1 - Planner: RECEIVED from s0036_inbox, SENT plan to s0036_tasks, COMMITTED.



Terminal 2 - Writer: RECEIVED from s0036_tasks (partition 1), SENT draft, COMMITTED.



Terminal 3 - Reviewer: RECEIVED draft, SENT status approved to s0036_final, COMMITTED.



Terminal 4 - sender: final JSON with the same correlation_id, status and answer.

The question "How should a team respond to a critical CVE found in a transitive dependency?" entered s0036_inbox with correlation_id fe8fdedd-0bb8-4cd1-990a-801e629d0b78. The Planner read it at offset 17 and published a 3 step plan to s0036_tasks (it landed on partition 1). The Writer turned the plan into a draft on s0036_drafts. The Reviewer ran its checks (non_empty, length_ok, on_topic were all true) and published status: "approved" with the answer to s0036_final. Every agent copied the correlation_id unchanged and committed its input offset only after its output was acknowledged by Kafka. The whole journey took about 34 s, most of it the Planner's LLM call (~28 s).

How Kafka helped the agents coordinate. Kafka was the only link between the agents: each one knows just its input and output topic, never another agent. That meant each agent could start, stop or crash on its own without breaking the others: if the Writer is down, plans simply wait in s0036_tasks until it comes back. Topics also acted as buffers, so a slow model call in one stage did not block the stage before it. Consumer groups gave us scaling for free: a second Writer in the same group shares the two partitions of s0036_tasks. Finally, every message is stored with its correlation_id, so the full journey of one question can be traced across all four topics.

Part 3 - Failure Behavior

Each failure was run twice by scripts/run_hw8_failures.py (make experiments-hw08): once with the fix switched off and once with it on. Per-process logs are saved in reports/hw08/raw/runs/.

3.1 Duplicate delivery and idempotency

```python
class ProcessedStore:
    """correlation_ids an agent has fully completed, persisted so a restart
    remembers them."""

    def __init__(self, group: str) -> None:
        STATE_DIR.mkdir(parents=True, exist_ok=True)
        self.path = STATE_DIR / f"{group}.json"
        self.done: set[str] = set(json.loads(self.path.read_text())) if self.path.
        exists() else set()

    def seen(self, correlation_id: str) -> bool:
        return correlation_id in self.done

    def mark(self, correlation_id: str) -> None:
        self.done.add(correlation_id)
        tmp = self.path.with_suffix(".tmp")
        tmp.write_text(json.dumps(sorted(self.done)))
        tmp.replace(self.path)  # atomic, so a crash never leaves a half-written
        file
```

```
try:
    if error:
        raise error
    if attempt == 1:
        log(component, "RECEIVED", topic=in_topic, correlation_id=cid,
        **where,
            kind=message.get("kind", "question"))
    if store is not None and cid and store.seen(cid):
        log(component, "DUPLICATE skipped (already processed)",
        correlation_id=cid, **where)
        output = None
        break
    if slow_seconds:
        log(component, "STARTED long processing", correlation_id=cid,
        seconds=slow_seconds, **where)
        time.sleep(slow_seconds)
    output = handle(message)
    if output is not None and out_topic:
        log(component, "SENT", topic=out_topic, correlation_id=cid,
            **{k: v for k, v in output.items() if k in ("plan",
            "draft", "status", "answer")})
        produce_json(producer, out_topic, output, key=output.get
        ("correlation_id"))
    if store is not None and cid:
        store.mark(cid)
```

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro runs % grep -h "RECEIVED\|SENT\|DUPLICATE" duplicate_before_fix/planner.log | cut -c1-170

2026-10-30T07:40:57.926-07:00 [planner] RECEIVED topic="s0036_inbox" correlation_id="773f4cfb-02fd-45ef-b923-fd6ea4042847" partition=0 offset=7 kind="question"
2026-10-30T07:40:57.927-07:00 [planner] SENT topic="s0036_tasks" correlation_id="773f4cfb-02fd-45ef-b923-fd6ea4042847" plan=["Define the key terms in: What makes a transi
2026-10-30T07:40:59.745-07:00 [planner] RECEIVED topic="s0036_inbox" correlation_id="773f4cfb-02fd-45ef-b923-fd6ea4042847" partition=0 offset=8 kind="question"
2026-10-30T07:40:59.745-07:00 [planner] SENT topic="s0036_tasks" correlation_id="773f4cfb-02fd-45ef-b923-fd6ea4042847" plan=["Define the key terms in: What makes a transi
((.venv) ) ayushishringi@Ayushis-MacBook-Pro runs %
```

2026-10-30T07:41:09.109-07:00 [planner] RECEIVED topic="s0036_inbox" correlation_id="9f4da386-b09b-4afe-bd22-dbcb776514db" partition=0 offset=9 kind="question"
2026-10-30T07:41:09.109-07:00 [planner] SENT topic="s0036_tasks" correlation_id="9f4da386-b09b-4afe-bd22-dbcb776514db" plan=["Define the key terms in: What makes a transitive dependency vulnerability hard to patch?", "Explain the ...
2026-10-30T07:41:10.935-07:00 [planner] RECEIVED topic="s0036_inbox" correlation_id="9f4da386-b09b-4afe-bd22-dbcb776514db" partition=0 offset=10 kind="question"
2026-10-30T07:41:10.936-07:00 [planner] DUPLICATE skipped (already processed) correlation_id="9f4da386-b09b-4afe-bd22-dbcb776514db" partition=0 offset=10

After fix (HW8_IDEMPOTENT=1) - log excerpt from reports/hw08/raw/runs/duplicate_after_fix/planner.log: the replay at offset 10 is skipped.

Before the fix the replayed message (offset 8) was processed again, giving two tasks and two final answers. After the fix the replay (offset 10) was logged as *DUPLICATE skipped* and only

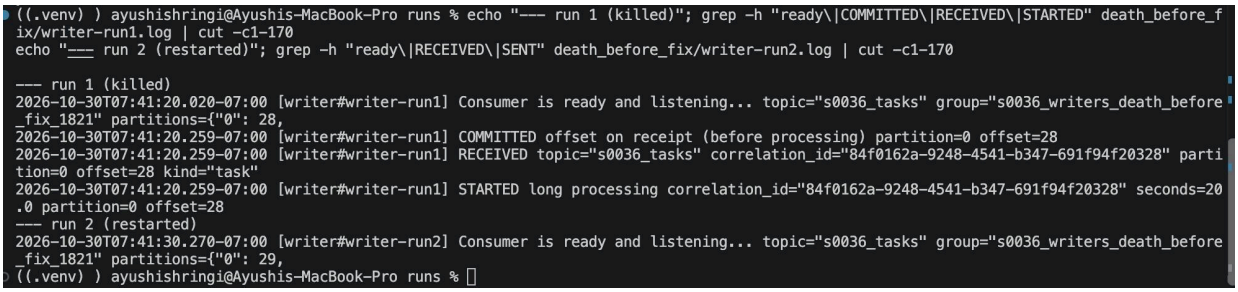
one final answer was produced.

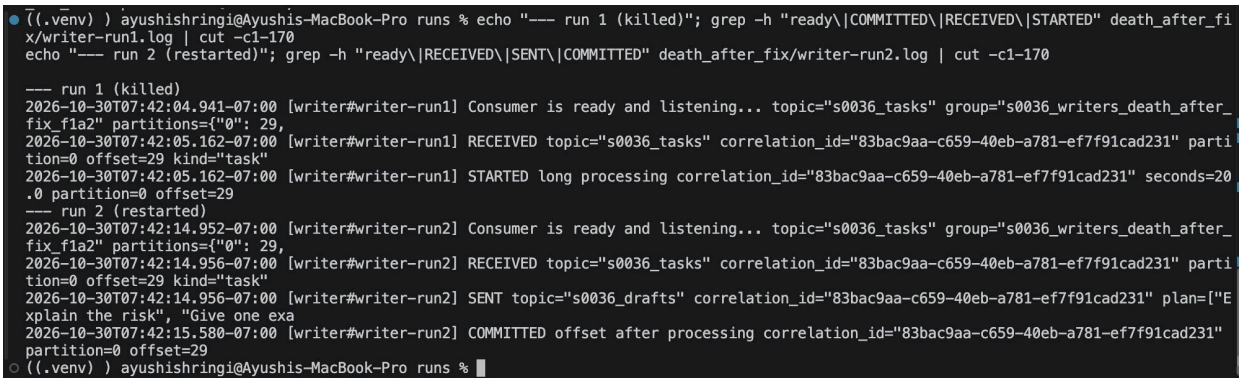
3.2 Consumer death and commit after success

```
    if commit_mode == "before":  # BEFORE FIX: offset committed as soon as the
    message is read
        consumer.commit(message=msg, asynchronous=False)
        log(component, "COMMITTED offset on receipt (before processing)",
        **where)

    attempt = 0
```

```
            break
    if commit_mode == "after":  # AFTER FIX: commit only once the output is
    safely in Kafka
        consumer.commit(message=msg, asynchronous=False)
        log(component, "COMMITTED offset after processing",
        correlation_id=cid, **where)
consumer.close()
```





Before the fix the offset was committed on receipt, so the restarted Writer started at 29 and the task at offset 28 was lost. After the fix nothing was committed before the kill, so the

restarted Writer received offset 29 again, produced the draft, then committed.

3.3 Poison message, bounded retries and dead letter

```
            store.mark(cid)
            break
        except Exception as exc:  # poison or transient failure
            log(component, "PROCESSING FAILED", correlation_id=cid,
            attempt=attempt, error=str(exc), **where)
            if retry_mode == "unbounded":  # BEFORE FIX: retry forever,
            blocking the partition
                time.sleep(1.0)
                continue
            if attempt < max_retries:
                time.sleep(0.5 * attempt)
                continue
            dead = {"correlation_id": cid, "failed_agent": name,
            "source_topic": in_topic, **where,
                    "attempts": attempt, "error": str(exc), "ts": now_iso(),
                    "original": message if message is not None else msg.value
                    ().decode("utf-8", "replace")}
            produce_json(producer, DEAD_LETTER_TOPIC, dead, key=cid)
            log(component, "MOVED to dead-letter topic",
            topic=DEAD_LETTER_TOPIC, correlation_id=cid,
                attempts=attempt, **where)
            break
```

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro runs % grep -h "RECEIVED\|FAILED\|MOVED\|SENT" poison_before_fix/writer.log | head -12 | cut -
c1-170
echo "failed attempts:"; grep -c "PROCESSING FAILED" poison_before_fix/writer.log

2026-10-30T07:42:19.992-07:00 [writer] RECEIVED topic="s0036_tasks" correlation_id="poison-fd6feef6" partition=1 offset=27 kind="task"
2026-10-30T07:42:19.993-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=1 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:20.998-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=2 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:21.999-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=3 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:23.003-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=4 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:24.005-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=5 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:25.006-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=6 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:26.009-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=7 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:27.015-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=8 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:28.018-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=9 error="plan must be a non-empty list of
 strings" partition=1 offset=27
2026-10-30T07:42:29.023-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=10 error="plan must be a non-empty list o
f strings" partition=1 offset=2
2026-10-30T07:42:30.025-07:00 [writer] PROCESSING FAILED correlation_id="poison-fd6feef6" attempt=11 error="plan must be a non-empty list o
f strings" partition=1 offset=2
failed attempts:
36
```

```
2026-10-30T07:42:55.981-07:00 [writer] RECEIVED topic="s0036_tasks" correlation_id="poison-5c8bb933" partition=1 offset=29
  kind="task"
2026-10-30T07:42:55.981-07:00 [writer] PROCESSING FAILED correlation_id="poison-5c8bb933" attempt=1 error="plan must be a non-
  empty list of strings" partition=1 offset=29
2026-10-30T07:42:56.483-07:00 [writer] PROCESSING FAILED correlation_id="poison-5c8bb933" attempt=2 error="plan must be a non-
  empty list of strings" partition=1 offset=29
2026-10-30T07:42:57.488-07:00 [writer] PROCESSING FAILED correlation_id="poison-5c8bb933" attempt=3 error="plan must be a non-
  empty list of strings" partition=1 offset=29
2026-10-30T07:42:57.766-07:00 [writer] MOVED to dead-letter topic topic="s0036_dead_letter" correlation_id="poison-5c8bb933"
  attempts=3 partition=1 offset=29
2026-10-30T07:42:57.769-07:00 [writer] RECEIVED topic="s0036_tasks" correlation_id="4b1d39bc-c037-4ee1-8270-a38e3ee00266"
  partition=1 offset=30 kind="task"
2026-10-30T07:42:57.769-07:00 [writer] SENT topic="s0036_drafts" correlation_id="4b1d39bc-c037-4ee1-8270-a38e3ee00266"
  plan=["Explain the risk", "Give one example", "List two fixes"] draft="Answer to 'What is an SBOM?': (1) Explai ...
```

After fix (HW8_RETRY=bounded, max 3) - log excerpt from reports/hw08/raw/runs/poison_after_fix/writer.log.

Before the fix the Writer retried forever (28 attempts in the 25 s measurement window; the log shows 36 because it kept retrying until it was stopped) and the good message behind it never ran. After the fix it tried 3 times, moved the message to s0036_dead_letter, and processed the good message at offset 30.

3.4 Ordering (VERIFY_SEED 260036, 20 messages, 2 partitions, 2 Writers)

```
331 def ordering_messages() -> list[dict]:
332  rng = random.Random(VERIFY_SEED)
333  seq = {e: 0 for e in ENTITIES}
334  messages = []
335  for index in range(20):
336   entity = rng.choice(ENTITIES)
337   seq[entity] += 1
338   probe_id = f"probe-{VERIFY_SEED}-{index:02d}"
339   messages.append({"correlation_id": probe_id, "probe_id": probe_id, "kind": "ordering_probe", "index": index,
340        "entity": entity, "seq": seq[entity],
341        "update": f"{entity} advisory update #{seq[entity]}"})
342  return messages
359   # no key: random partition per message (sticky batching off); key: hash(entity) -> one partition
360   producer = make_producer(**({} if keyed else {"sticky.partitioning.linger.ms": 0}))
361   sent = []
362   for m in messages:
363     meta = produce_json(producer, TASKS_TOPIC, m, key=m["entity"] if keyed else None)
364     sent.append({**m, **meta})
365   say(f"SENT 20 messages ({'key=entity' if keyed else 'no key'})")
```

scripts/run_hw8_failures.py - 20 seeded messages; run 1 sends with no key, run 2 with key = entity.

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % sed -n 186,222p reports/hw08/RUN_LOG.txt | cut -c1-120


================================================================================================
Part 3 ordering - run1_no_key: 20 messages, s0036_tasks has 2 partitions, 2 Writers in one group  (2026-10-30T07:43:00)
================================================================================================
2026-10-30T07:43:00.847-07:00 [orchestrator] started writer-A (pid 80150) env={'HW8_LLM': '0', 'HW8_GROUP_SUFFIX': '_ord
2026-10-30T07:43:00.851-07:00 [orchestrator] started writer-B (pid 80151) env={'HW8_LLM': '0', 'HW8_GROUP_SUFFIX': '_ord
2026-10-30T07:43:05.160-07:00 [orchestrator] writer-A assignment: {"0":
2026-10-30T07:43:05.160-07:00 [orchestrator] writer-B assignment: {"1":
2026-10-30T07:43:05.184-07:00 [orchestrator] SENT 20 messages (no key)

order writer    partition entity      seq sent_index
    1 writer-B          1 log4j-core    1 2
    2 writer-A          0 openssl       1 0
    3 writer-A          0 lodash        1 1
    4 writer-B          1 urllib3       1 3
    5 writer-A          0 lodash        2 5
    6 writer-B          1 urllib3       2 4
    7 writer-A          0 lodash        3 6
    8 writer-A          0 lodash        4 7
    9 writer-B          1 lodash        5 9
   10 writer-A          0 openssl       2 8
   11 writer-B          1 log4j-core    2 10
   12 writer-B          1 lodash        6 12
   13 writer-A          0 openssl       3 11
   14 writer-A          0 urllib3       3 15
   15 writer-B          1 openssl       4 13
   16 writer-A          0 urllib3       4 17
   17 writer-A          0 openssl       5 18
   18 writer-B          1 lodash        7 14
   19 writer-A          0 log4j-core    4 19
   20 writer-B          1 log4j-core    3 16

per entity (processing order of seq, partitions used):
  openssl     seq=[1, 2, 3, 4, 5] partitions=[0, 1] in order
  lodash      seq=[1, 2, 3, 4, 5, 6, 7] partitions=[0, 1] in order
  log4j-core  seq=[1, 2, 4, 3] partitions=[0, 1] OUT OF ORDER
  urllib3     seq=[1, 2, 3, 4] partitions=[0, 1] in order
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Run 1, no key: log4j-core processed 1, 2, 4, 3 (OUT OF ORDER).

2026-10-30T07:43:10.978-07:00 [orchestrator] started writer-A (pid 80201) env={'HW8_LLM': '0', 'HW8_GROUP_SUFFIX': '_order_run2_key_entity_726c'}
2026-10-30T07:43:10.980-07:00 [orchestrator] started writer-B (pid 80202) env={'HW8_LLM': '0', 'HW8_GROUP_SUFFIX': '_order_run2_key_entity_726c'}
2026-10-30T07:43:15.288-07:00 [orchestrator] writer-A assignment: {"1":
2026-10-30T07:43:15.288-07:00 [orchestrator] writer-B assignment: {"0":
2026-10-30T07:43:15.311-07:00 [orchestrator] SENT 20 messages (key=entity)

Run 2, key = entity - setup lines from reports/hw08/RUN_LOG.txt.

order	writer	partition	entity	seq	sent_index
1	writer-B	0	lodash	1	1
2	writer-A	1	openssl	1	0
3	writer-B	0	log4j-core	1	2
4	writer-A	1	urllib3	1	3
5	writer-B	0	lodash	2	5
6	writer-A	1	urllib3	2	4
7	writer-B	0	lodash	3	6
8	writer-B	0	lodash	4	7
9	writer-A	1	openssl	2	8
10	writer-B	0	lodash	5	9
11	writer-A	1	openssl	3	11
12	writer-B	0	log4j-core	2	10
13	writer-A	1	openssl	4	13
14	writer-A	1	urllib3	3	15
15	writer-B	0	lodash	6	12
16	writer-A	1	urllib3	4	17
17	writer-B	0	lodash	7	14
18	writer-A	1	openssl	5	18
19	writer-B	0	log4j-core	3	16
20	writer-B	0	log4j-core	4	19

per entity (processing order of seq, partitions used):
openssl seq=[1, 2, 3, 4, 5] partitions=[1] in order
lodash seq=[1, 2, 3, 4, 5, 6, 7] partitions=[0] in order
log4j-core seq=[1, 2, 3, 4] partitions=[0] in order
urllib3 seq=[1, 2, 3, 4] partitions=[1] in order

Run 2, key = entity - processing order from reports/hw08/RUN_LOG.txt: every entity on one partition, all in order.

Without a key, messages for one entity were spread over both partitions, so log4j-core was processed as 1, 2, 4, 3. With key = entity, each entity stayed on one partition (openssl p1, lodash p0, log4j-core p0, urllib3 p1) and all four were processed in order.

Failure Table

Failure	Before fix	After fix
Duplicate delivery	2 tasks, 2 final answers for one question	1 task, 1 final; replay logged "DUPLICATE skipped"
Consumer killed mid-work	Offset committed on receipt (29 vs task at 28); after restart, not redelivered, 0 drafts, so the task was lost	Offset not committed (29 = task offset); after restart it was redelivered, 1 draft, committed 30
Poison message	28 attempts in 25 s, never dead-lettered, good message blocked	3 attempts s0036_dead_letter, good message processed
Ordering, 2 partitions / 2 consumers	No key: log4j-core processed [1, 2, 4, 3]	key = entity: 0 out of order, each entity on one partition

Comparison with HW2

Benefits gained:

1. Failure isolation: when the Writer was killed, the Planner and Reviewer kept running and the task waited in s0036_tasks until the Writer came back. In HW2, one exception stopped the whole graph.

2. Independent scaling: a second Writer joined the same consumer group and Kafka split the two partitions between them, with no change to the other agents.

New responsibilities:

1. Duplicates: Kafka delivers at least once, so each agent must be idempotent (here, a saved set of completed correlation_ids).

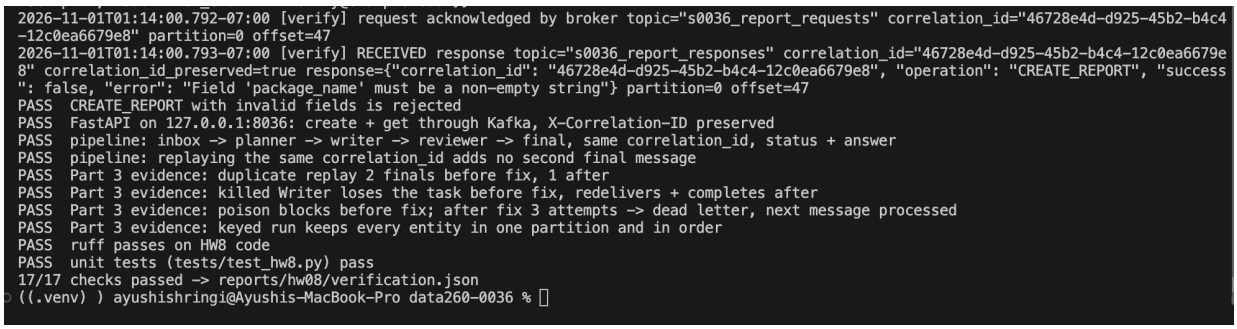
2. Offsets, retries and ordering: commit only after the output is in Kafka, cap retries and use a dead-letter topic so one bad message can't block a partition, and use a key when order matters. In HW2, a function call just returned in order, exactly once.

Verification

make verify-hw08: 17/17 checks passed; results written to reports/hw08/verification.json (commit and tag_hw8_commit 6a17a98).



```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % clear
make verify-hw08
PYTHONPATH=. .venv/bin/python3 scripts/verify_hw8.py
PASS  docker: Kafka, worker and FastAPI containers running
PASS  worker log contains 'Consumer is ready and listening...'
PASS  all 7 s0036_* topics exist with the expected partition counts
2026-11-01T01:13:59.775-07:00 [verify] SENT request topic="s0036_report_requests" correlation_id="c589d4a0-b3cb-40cc-8bde-51e11493b9b9" mes
sage={"correlation_id": "c589d4a0-b3cb-40cc-8bde-51e11493b9b9", "reply_to": "s0036_report_responses", "operation": "CREATE_REPORT", "data":
 {"package_name": "verify-hw08-260036", "affected_version": "< 1.0.0", "severity": "low", "description": "Temporary report created by the H
W8 verification script.", "submitter_email": "verify@example.edu"}}
2026-11-01T01:14:00.779-07:00 [verify] request acknowledged by broker topic="s0036_report_requests" correlation_id="c589d4a0-b3cb-40cc-8bde
-51e11493b9b9" partition=0 offset=43
2026-11-01T01:14:00.785-07:00 [verify] RECEIVED response topic="s0036_report_responses" correlation_id="c589d4a0-b3cb-40cc-8bde-51e11493b9b
9" correlation_id_preserved=true response={"correlation_id": "c589d4a0-b3cb-40cc-8bde-51e11493b9b9", "operation": "CREATE_REPORT", "success
": true, "report_id": "05b8964d-9cd9-4423-a05e-e1fedea38621", "message": "Report created"} partition=0 offset=43
PASS  CREATE_REPORT over Kafka returns success + UUID report_id, correlation_id preserved
2026-11-01T01:14:00.785-07:00 [verify] SENT request topic="s0036_report_requests" correlation_id="9985cfe5-3cb2-44c4-9d98-9044c6a757ab" mes
sage={"correlation_id": "9985cfe5-3cb2-44c4-9d98-9044c6a757ab", "reply_to": "s0036_report_responses", "operation": "GET_REPORT", "data": {"
report_id": "05b8964d-9cd9-4423-a05e-e1fedea38621"}}
2026-11-01T01:14:00.786-07:00 [verify] request acknowledged by broker topic="s0036_report_requests" correlation_id="9985cfe5-3cb2-44c4-9d98
-9044c6a757ab" partition=0 offset=44
2026-11-01T01:14:00.787-07:00 [verify] RECEIVED response topic="s0036_report_responses" correlation_id="9985cfe5-3cb2-44c4-9d98-9044c6a757a
b" correlation_id_preserved=true response={"correlation_id": "9985cfe5-3cb2-44c4-9d98-9044c6a757ab", "operation": "GET_REPORT", "success":
true, "report": {"report_id": "05b8964d-9cd9-4423-a05e-e1fedea38621", "package_name": "verify-hw08-260036", "affected_version": "< 1.0.0",
"severity": "low", "description": "Temporary report created by the HW8 verification script.", "submitter_email": "verify@example.edu"}} par
tition=0 offset=44
PASS  GET_REPORT round-trips the created report, correlation_id preserved
2026-11-01T01:14:00.787-07:00 [verify] SENT request topic="s0036_report_requests" correlation_id="7f7240a3-ddd9-49ca-870c-82afd52f4cb3" mes
sage={"correlation_id": "7f7240a3-ddd9-49ca-870c-82afd52f4cb3", "reply_to": "s0036_report_responses", "operation": "GET_REPORT", "data": {"
report_id": "ce1542b3-8bcd-4476-b9e5-ca8b6b4b13b4"}}
2026-11-01T01:14:00.788-07:00 [verify] request acknowledged by broker topic="s0036_report_requests" correlation_id="7f7240a3-ddd9-49ca-870c
-82afd52f4cb3" partition=0 offset=45
2026-11-01T01:14:00.789-07:00 [verify] RECEIVED response topic="s0036_report_responses" correlation_id="7f7240a3-ddd9-49ca-870c-82afd52f4cb
3" correlation_id_preserved=true response={"correlation_id": "7f7240a3-ddd9-49ca-870c-82afd52f4cb3", "operation": "GET_REPORT", "success":
false, "error": "Report not found"} partition=0 offset=45
PASS  GET_REPORT unknown id -> success false + error, correlation_id preserved
2026-11-01T01:14:00.789-07:00 [verify] SENT request topic="s0036_report_requests" correlation_id="ac337a40-c4c6-4928-8b97-e656297eae5e" mes
sage={"correlation_id": "ac337a40-c4c6-4928-8b97-e656297eae5e", "reply_to": "s0036_report_responses", "operation": "DROP_REPORT", "data": {
}}
2026-11-01T01:14:00.790-07:00 [verify] request acknowledged by broker topic="s0036_report_requests" correlation_id="ac337a40-c4c6-4928-8b97
-e656297eae5e" partition=0 offset=46
2026-11-01T01:14:00.791-07:00 [verify] RECEIVED response topic="s0036_report_responses" correlation_id="ac337a40-c4c6-4928-8b97-e656297eae5
e" correlation_id_preserved=true response={"correlation_id": "ac337a40-c4c6-4928-8b97-e656297eae5e", "operation": "DROP_REPORT", "success":
 false, "error": "Unknown operation"} partition=0 offset=46
PASS  unknown operation -> success false + error, correlation_id preserved
2026-11-01T01:14:00.791-07:00 [verify] SENT request topic="s0036_report_requests" correlation_id="46728e4d-d925-45b2-b4c4-12c0ea6679e8" mes
sage={"correlation_id": "46728e4d-d925-45b2-b4c4-12c0ea6679e8", "reply_to": "s0036_report_responses", "operation": "CREATE_REPORT", "data":
 {"package_name": "", "affected_version": "< 1.0.0", "severity": "urgent", "description": "Temporary report created by the HW8 verification
 script.", "submitter_email": "verify@example.edu"}}
2026-11-01T01:14:00.792-07:00 [verify] request acknowledged by broker topic="s0036_report_requests" correlation_id="46728e4d-d925-45b2-b4c4
-12c0ea6679e8" partition=0 offset=47
```



make verify-hw08 - 17/17 checks passed.

AI Use

1. What I used AI for and what I did myself. I used an AI coding assistant to help write and debug the Kafka worker, client, FastAPI wrapper, the three agents and the failure-test scripts. I ran the Docker stack, the demo, the agents in separate terminals and the failure tests myself, took the screenshots, checked the results against the assignment, and wrote the report.

2. One AI output that was wrong. The first version of the worker printed "Consumer is ready and listening..." after a loop that polled Kafka until the consumer got its partition. Any request that arrived during that loop was read and thrown away, so it would never get a response.

3. How I found it. Reading the worker code, I saw that the messages returned by that waiting loop were ignored. A request sent while the worker was still starting would have timed out on the client side.

4. What I changed. The ready message is now logged from the Kafka on_assign callback, so the worker never polls a message it does not process. Reply readers instead assign their partition at an explicit offset (the current end), so a response produced right after the request is always seen.